COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structure and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	09/09/26	**DATE SUBMITTED:**	09/16/26-
NAME:	Nieles, Cenndy M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 Hands-on Activity Sorting Algorithm Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

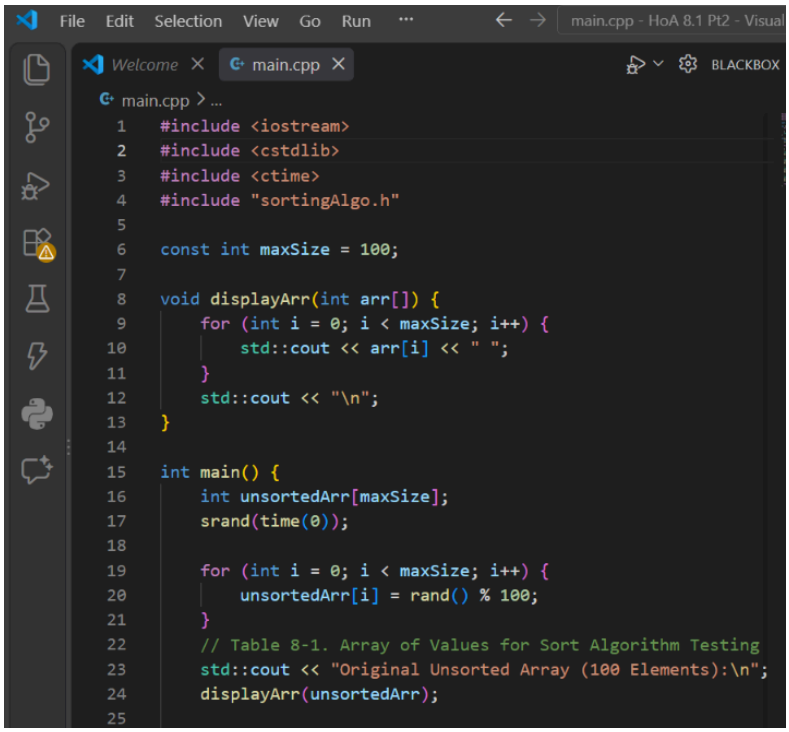
Code + Console Screenshot	
Observations	allocates an array of size 100 maxSize = 100 to serve as the baseline test dataset. The displayArr functions correctly through the array and outputs all 100 elements to the console to randomize unsorted arrays to be passed into various sorting algorithms for performance and to testing.

Table 8-1. Array of Values for Sort Algorithm Testing

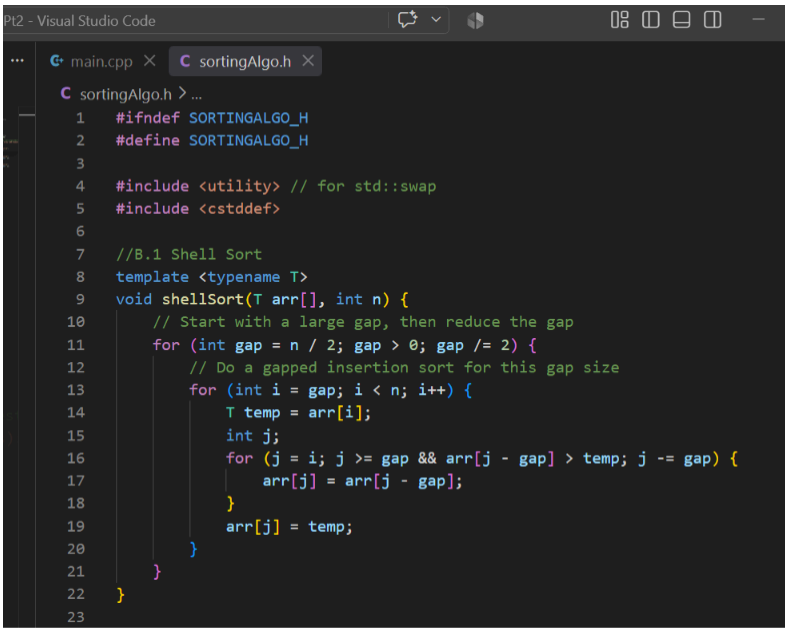
Code + Console Screenshot	 SortingAlgorithm.h  main.cpp
Observations	The header file sortingAlgo.h implements Shell Sort using a template function (shellSort) that takes a gapped insertion approach, starting with then follow the sorting execution displayArr(unsorted) is called to print the new order of elements to the consoled under the heading shell sort .

Table 8-2 . Shell Sort Technique

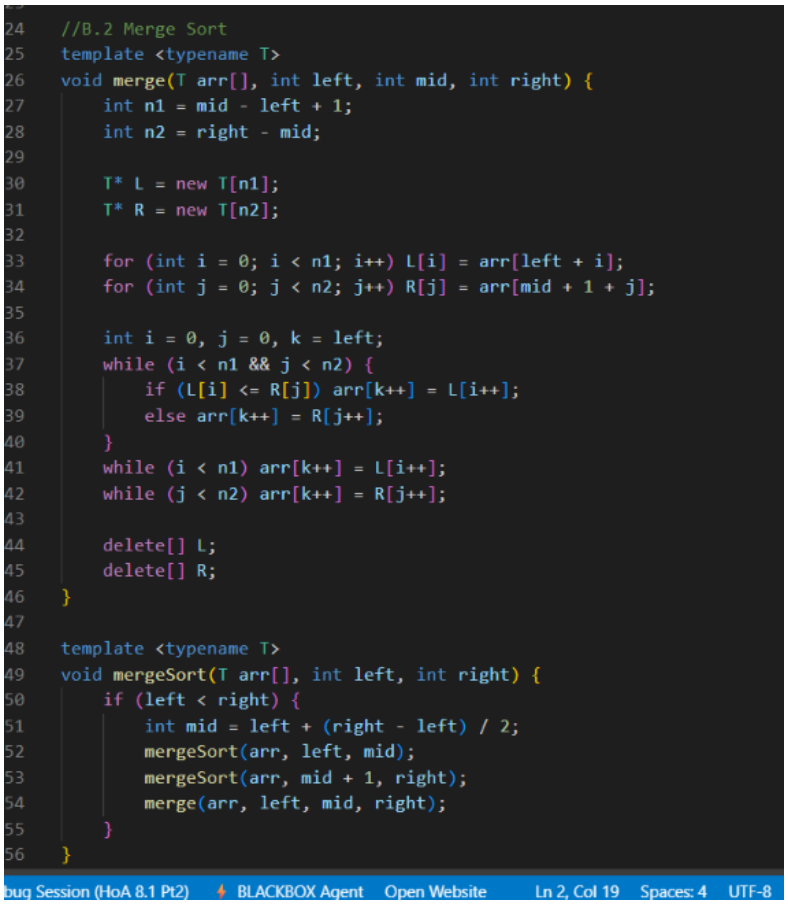
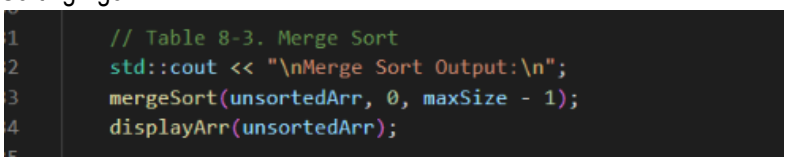
Code + Console Screenshot	 SortingAlgo.h 
Observations	This program implements Merge Sort, which splits the arrays into smaller halves until each piece is small, sorts them, and then merges them back together. The two functions inside the SortingAlgo.h - merge sort to split and to merge and combine the sorted pieces .

Table 8-3. Merge Sort Algorithm

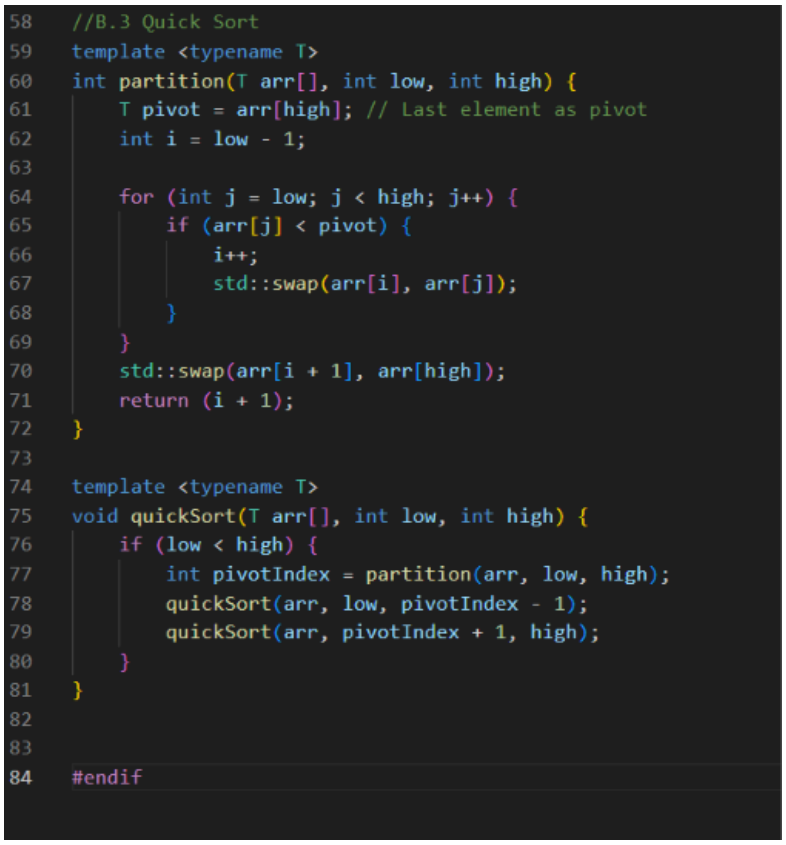
Code + Console Screenshot	 SortingAlgorithm.h  main.cpp
Observation	The code picks the last element as a "pivot" and reorganizes the array so that smaller numbers are moved to the left and larger numbers are moved to the right. It uses a recursive quickSort function to continuously break down and sort the left and right sub-lists independently. program applies the Quick Sort algorithm to the random dataset and prints the fully sorted elements to the console.

Table 8-4. Quick Sort Algorithm

```
58  //B.3 Quick Sort
59  template <typename T>
60  int partition(T arr[], int low, int high) {
61      T pivot = arr[high]; // Last element as pivot
62      int i = low - 1;
63
64      for (int j = low; j < high; j++) {
65          if (arr[j] < pivot) {
66              i++;
67              std::swap(arr[i], arr[j]);
68          }
69      }
70      std::swap(arr[i + 1], arr[high]);
71      return (i + 1);
72  }
73
74  template <typename T>
75  void quickSort(T arr[], int low, int high) {
76      if (low < high) {
77          int pivotIndex = partition(arr, low, high);
78          quickSort(arr, low, pivotIndex - 1);
79          quickSort(arr, pivotIndex + 1, high);
80      }
81  }
82
83
84  #endif
```

```
36      // Table 8-4. Quick Sort
37      std::cout << "\nQuick Sort Output:\n";
38      quickSort(unsortedArr, 0, maxSize - 1);
39      displayArr(unsortedArr);
40
41      return 0;
42  }
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, absolutely. Once the partition method in Quick Sort separates an array into a left sub-list and a right sub-list around a pivot, those sub-lists become completely independent arrays. You can use any sorting algorithm (such as Insertion Sort, Selection Sort, or Merge Sort) to sort them. In fact, many professional implementation like C++ std::sort or Introsort) switch to Insertion Sort for small sub-lists because it is faster for small numbers of elements.

Demonstration Example: Initial array: {24, 9, 29, 14, 45, 18}
Step 1 (Partition around pivot = 18): Rearranging elements around 18 leaves: Left Partition: {9, 14} | Pivot: [18] | Right Partition: {45, 29, 24}
Step 2 (Sort partitions using another algorithm): - Apply Insertion Sort to the Left Partition {9, 14} -> {9, 14} - Apply Selection Sort to the Right Partition {45, 29, 24} -> {24, 29, 45}
Step 3 (Recombine): Connecting Left + Pivot + Right yields the fully sorted array: {9, 14, 18, 24, 29, 45}.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Fastest algorithm for this array: Quick Sort (or else Insertion Sort or Shell Sort if array size is considered). Quick Sort will give the fastest runtime for an unsorted, degenerate 15 element array. This is because it works in place with no use of temporary heap or buffer space (unlike Merge Sort). The inner comparison and swapping loops will have least overhead with efficient CPU cache access. Arrays of this size (N = 15) run in essentially zero overhead due to Insertion Sort and Shell Sort. Why Merge Sort and Quick Sort achieve O(N log N) time complexity: Both sorting methods depend on the Divide and Conquer strategy that separates the sorting process into two components from the structural point of view: * The log N Factor (Number of Tree Levels / Number of Divisions): To split a collection of size N into equal parts multiple times results in building a balanced binary decomposition tree. It will take log_2 N number of steps to divide N down to one-element cases when N = 1. * The N Factor (Linear Work on Each Level): In each level of decomposition tree all N elements have to be analyzed once: * In the case of Merge Sort, the comparison and merge of sub-arrays on each level will require a linear number of operations equal to O(N). * In the case of Quick Sort, the partitioning of the array by the pivot on each level will require the analysis of each element once, thus resulting in O(N) number of operations. comparisons. Multiplication of linear number of operations on each level (N) by the number of levels (log N) will give overall complexity of O(N * log N).

Conclusion

This lab activity provided hands-on experience about how Shell Sort, Merge Sort, and Quick Sort work and perform in C++. Running all three sorts on a random 100 element array helped me understand how these algorithms work around their $O(n^2)$ limitations. Shell Sort used optimized insertion sort, where distant elements were compared within decreasing gap intervals to pre-sort the array. In addition, Merge Sort and Quick Sort demonstrated the advantages of divide-and-conquer approach: Merge Sort ensured guaranteed $O(n \log n)$ running time through recursive dividing and merging of an array, while Quick Sort accomplished fast in-place sorting based on pivot element. The supplementary activity showed that sub-lists could be sorted individually using some other methods and mathematically confirmed the $O(n \log n)$ complexity by multiplying linear-level operations with logarithmic-level number of divisions. Overall, I did well in working with templates, manipulating dynamically allocated arrays during merging, and tracing recursive calls. The aspect that should be worked on next is implementing a pivot selection by median-of-three method in Quick Sort to avoid degradation in worst cases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Gemini to guide and explain the part that I don't understand**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.